ddos_resilience

DDoS Resilience Testing

Revision: 5 **Last modified:** 2026-08-26T12:00:00Z

Documents `challenges/scripts/ddos_resilience_challenge.sh` — the DDoS-class testing scaffold added by **BOB-074** to close a gap in boba's §11.4.27 mandated test-type matrix (see `docs/testing/test_type_matrix.md` for the full audit). §11.4.85 (stress + chaos mandate) and §11.4.27 (100% test-type coverage, DDoS is one of the enumerated types) are the governing anchors.

What it tests

boba exposes three public, unauthenticated-by-default HTTP surfaces:

Endpoint	Port	Probed path	Why it matters
Merge search (FastAPI/ uvicorn)	7187	`GET /health`	Fans out to up to 43 trackers on `POST /api/v1/search` — the expensive, real DDoS target (see "Scoping decisions" below for why the fanout path itself is never driven)
qBittorrent WebUI	7185	`GET /`	`network_mode: host`, directly reachable from the host network
	7189	`GET /healthz`	Owns Jackett credentials +

Endpoint	Port	Probed path	Why it matters
Boba Jackett API (Go/Gin)			indexer overrides, backed by encrypted SQLite

For each reachable endpoint the challenge drives a bounded, localhost-only curl-parallel-loop (primary — tool-independent) plus `ab`/ApacheBench (supplementary evidence, when installed) across three concurrency tiers, and asserts:

1. **Crash resistance** — zero 5xx responses and zero connection failures (including client-side timeouts under load — see "Terminology" below) across all three tiers.
2. **Rate limiting** — a 429 (or equivalent) appears once load is heavy enough, **per endpoint**. See "RED/GREEN polarity" below. **Superseded 2026-08-26:** the 2026-08-18 finding that no rate limiter existed anywhere in the stack is no longer true — `merge_search :7187` now enforces one (`x-ratelimit-limit: 120` with a `retry-after`; measured live 33 x 429 at c=50), so it **PASSes** in GREEN and would FAIL in RED — exactly the polarity flip the original clause predicted. `:7185` and `:7189` still `SKIP` as `extension_absent`.
3. **Cross-endpoint isolation** — while one endpoint is under its heaviest tier, the other two stay **responsive** within a bounded timeout. "Responsive" is **any 2xx, OR a 429 carrying a well-formed `Retry-After`** (BOB-163). "Degraded" is a connection failure, a timeout, any 5xx, any other non-2xx, and a 429 whose `Retry-After` is **absent or malformed**. See "Detector 3" below for why, and for the two residual risks this assertion does **not** close.

Terminology: "crash" includes client-side timeout

The task brief's crash-resistance criterion is "return 5xx or connection-reset." This challenge additionally counts a client-side `--max-time` timeout (curl exit code 28) as a crash-resistance failure. From the caller's perspective, a backend that never answers within a reasonable window under load is exactly as unavailable as one that resets the connection — and (see "Findings" below) this distinction is what caught a real defect. The per-tier output breaks timeouts out separately (`timeout=N other=M`) so the two failure modes are never conflated when reading results.

Bounded chaos knobs (§12.6 / §12.11 host-safety)

Knob	Value	Rationale
Concurrency tiers	10, 25, 50	Modest ramp; `ab` refuses `-c` > `-n`, so `REQUESTS_PER_TIER` must be ≥ the heaviest tier
Requests per tier	50	Matches the heaviest concurrency tier (see above)
Per-request timeout	3s	curl `--max-time`; also fed to `ab -s`
Per-tier wall-clock budget	12s	Hard early-stop — a genuinely slow backend gets a **smaller sample**, never a **longer-running script** (see `run_curl_status_probe`)
Cross-endpoint probe timeout	3s	
Scope	`127.0.0.1` only, hardcoded	No host override exists — this challenge cannot be pointed at anything but localhost

Measured wall-clock for a full live run (2026-08-18, 8-core/62G dev host, stack already running): **~17s** when all three endpoints are warm, **~55s** in the run that caught the boba-jackett cold-start finding below. Both are comfortably inside `run_all_challenges.sh`'s 180s per-script timeout.

The challenge never targets anything but `127.0.0.1`, never issues a destructive command, and its `--self-validate` mode exercises six THROWAWAY local `python3` HTTP servers — never the real boba stack. (Two when this line was written; the rate-limit detector added three in BOB-114 and the sibling detector one in BOB-163, the last serving all nine of its cases from one process via distinct paths.)

RED/GREEN polarity (§11.4.115)

The **rate-limiting** assertion is the one with real, meaningful polarity today (the crash-resistance and cross-endpoint checks are real pass/fail checks regardless of `RED_MODE` — see “Findings” for what they actually caught).

- **`RED_MODE=1`** (reproduce the defect): asserts the **absence** of any 429 response under the heaviest tier. **State as of 2026-08-26 (measured, not assumed):** it now `PASS`es on `:7185` and `:7189` only, and **`FAIL`s on `merge_search :7187`** — that endpoint enforces a limiter (`x-ratelimit-limit: 120`, 32–33 × 429 at c=50), and the detector’s own truth table maps `engaged` + `RED_MODE=1` → `FAIL`. That FAIL is the designed polarity flip, not a regression: RED reproduces a defect that no longer exists on `:7187`. *(Superseded: through 2026-08-18 this read “`PASS`es for all three endpoints … no rate-limit middleware, no `slowapi`-style Python limiter, no throttle in either Go service, and no nginx/reverse-proxy layer exists anywhere in the stack”, grepped across `download-proxy/src/`, `qBitTorrent-go/internal/` and `docker-compose.yml`. True when written; false now.)*
- **`RED_MODE=0`** (default, GREEN guard): where a limiter **is** present the 429s are asserted and `PASS` (`:7187` today); where it is absent the same absence-of-429 observation is honestly `SKIP`ped (reason `extension_absent`) rather than bluffed as a `PASS` or unfairly marked `FAIL` — §11.4.6 forbids inventing a threshold nothing in the codebase enforces.

Real polarity going forward: the day a rate limiter is configured for any of the three endpoints, `RED_MODE=0` starts asserting 429s actually appear under load (a genuine `PASS`/`FAIL` on whether the configured limit works), and `RED_MODE=1` starts `FAIL`ing (correctly signalling the defect this anchor targets has been fixed). If that rate limiter is later stripped again, `RED_MODE=1` goes back to `PASS` (defect reproduced) and `RED_MODE=0` would `FAIL` — “strip rate limiting → challenge FAILs” holds from the point a limiter first exists.

Self-validation (§11.4.107(10))

The self-validation runs **automatically at the start of every invocation**, including the bare no-arg form `challenges/scripts/run_all_challenges.sh` uses — it is not something an operator has to remember to ask for. If a detector fails its own fixtures the challenge refuses to run the live probes at all and exits `1`: an unvalidated detector mints no verdicts (§11.4.115(F)). Registration is not coverage (§11.4.226) — before **BOB-114** the

self-test existed but nothing in the normal path ever executed it. `--self-validate` still works as a standalone mode for running just the fixtures.

Every fixture is a throwaway local `python3 ThreadingHTTPServer` bound to port **0** (the kernel picks a free port, which the server prints on stdout and the script reads back) — never a fixed port, so two concurrent runs cannot collide, and never the real boba stack.

Detector 1 — crash resistance

Fixture	Behaviour	Detector must
golden-**good**	always `200`	not flag it
golden-**bad**	always `500`	see every response as `5xx`

Detector 2 — rate limiting (added by BOB-114)

Before BOB-114 this detector had **no fixture at all** and had therefore never been observed to FAIL — unvalidated instrumentation in the §11.4.115(F) sense. The RED that proved it: a §1.1 mutation making the 429 tally always report `999` (so the detector always claims "rate limiting engaged") left the old self-test **fully green**. A detector that would certify a completely unprotected deployment as rate-limited was invisible to its own self-test.

Fixture	Behaviour	Detector must classify
golden-**good** (`enforcing`)	`200` up to a threshold, `429` past it	`engaged`
golden-**bad** (`absent`)	always `200`; never `429`, never `503`, no matter the burst	`absent`
golden-**FALSE-with-carrier** (`carrier`)	always `200`, but carrying `X-RateLimit-Limit` / `X-RateLimit-Remaining` / `X-RateLimit-Policy` / `Retry-After` headers **and** a body that literally reads `429 Too Many Requests: rate limit exceeded`	`absent`

The carrier fixture is the §11.4.201(7)(a) guard: a server that *advertises* a limit it never enforces is the exact lookalike a substring-matching detector false-fires on, and a false-positive refusal is as forbidden as a false pass (§11.4.201(1)). The detector matches on **field 1 of the probe log — the curl-reported HTTP status code** — never on bodies or headers.

The self-test also asserts the full **(class × `RED_MODE`) verdict truth table**, so both polarity arms are exercised rather than only the one boba's current state happens to hit:

class	`RED_MODE=0` (GREEN)	`RED_MODE=1` (RED)
`engaged`	`PASS`	`FAIL`
`absent`	`SKIP` (`extension_absent`)	`PASS`

Detector 3 — sibling responsiveness (added by BOB-163)

Assertion (c)'s oracle previously had **no fixture at all** and had therefore never been observed to FAIL on a genuinely broken artifact — unvalidated instrumentation in the §11.4.115(F) sense, exactly the gap BOB-114 closed for detector 2.

Why the rule changed. Assertion (c) required a sibling probe to answer `^2`. `:7187` now enforces a real limiter (measured live 2026-08-26: `x-ratelimit-limit: 120`, `retry-after: 45`, `server: uvicorn`). A full run sends ~250 requests to `:7187`, so a sibling probe fired while *another* endpoint is under its heaviest tier lands on an exhausted bucket and gets a 429 — which the `^2` rule read as "endpoint degraded". That is a §11.4.1 **FAIL-bluff**: it condemns a service that is working correctly and protecting itself as designed.

Operator decision (§11.4.66, 2026-08-26): *"Yes, if Retry-After is well-formed"* — a 429 carrying a valid `Retry-After` is evidence of a live service correctly protecting itself.

Well-formed means parsed, not present.
`retry_after_wellformed` accepts a **positive** `1*DIGIT` delta-seconds (RFC 9110 §10.2.3 — so `0` and `-5` are rejected) or an HTTP-date matched **structurally** as IMF-fixdate / RFC 850 / asctime (RFC 9110 §5.6.7 requires a *recipient* to accept all three; refusing a legal obsolete form would be its own §11.4.201(1) false positive) and then **semantically parsed** via `date -d`, which rejects a correctly-shaped but impossible date. `HAVE_DATE_PARSE` is itself a §11.4.201(7)(b) control needle: a known-good IMF-fixdate is run through the parser at load, and if it fails the host has no parser,

the semantic step is skipped, and the fixture that depends on it reports `NOT EXERCISED (reason=no_date_parser_on_host)` rather than accusing the detector.

Fixture	`Retry-After` sent	Detector must classify
`ok200`	— (status 200)	`responsive`
`r429_valid_seconds`	`30`	`responsive`
`r429_valid_httpdate`	`Wed, 21 Oct 2015 07:28:00 GMT`	`responsive`
`r429_missing`	*header absent*	`degraded`
`r429_garbage`	`soon-ish`	`degraded`
`r429_zero`	`0`	`degraded`
`r429_neg`	`-5`	`degraded`
`r429_baddate`	`Sun, 32 Nov 1994 25:99:99 GMT`	`degraded` (skipped honestly where `date -d` is absent)
`srv500`	— (status 500)	`degraded`
`refused`	— (nothing listening)	`degraded`

The four rows `r429_garbage` / `r429_zero` / `r429_neg` / `r429_baddate` are **golden-bad-with-carrier**: the header *is* present on every one of them, so a detector that merely checks for a non-empty header passes them all. Only a real parse separates them — which is what the M1 mutation below flips.

The **decision** is fixture-covered too, not just the classification: `sibling_isolation_report` takes an `<expected-count>` — how many sibling probes were LAUNCHED — and may report “isolation held” only when it parsed an observation for **every** probe it fired. It is driven through nine polarities:

Decision fixture	Log shape	Expect
`all-responsive`	3 real responsive observations	`rc=0`
`one-degraded`	5 observations, 2 of them degraded	`rc=1`
`empty-log`	0 bytes	`rc=1`, `NO SIBLING OBSERVATIONS`
`blank-line`	1 byte, one empty line	`rc=1`, `NO SIBLING OBSERVATIONS`
`whitespace-only`	6 bytes, no parseable field	`rc=1`, `NO SIBLING OBSERVATIONS`
`torn-degraded`		

Decision fixture	Log shape	Expect
	one degraded observation, trailing newline lost	`rc=1`, and it must name the sibling `DEGRADED`
`torn-mixed`	one responsive + one torn degraded	`rc=1`, and it must name the sibling `DEGRADED`
`partial`	1 observation where 2 probes fired	`rc=1`, `INCOMPLETE SIBLING OBSERVATIONS`
`complete-2of2`	2 observations, 2 probes	`rc=0` — the false-positive guard

The last row is load-bearing: a count contract that refuses a *complete* log would be the §11.4.201(1) false-positive refusal, as forbidden as a false pass. The two torn rows assert `DEGRADED` rather than the count message on purpose — that is what proves the torn observation was **parsed** (via the `|| [ -n "$sname" ]` guard) rather than merely counted-missing; without the guard they would still return `rc=1`, but for the wrong reason.

Why this is wider than a zero-byte check (§11.4.201(6)): a log can be non-empty and still carry fewer parsed observations than probes launched. A blank or whitespace-only log has bytes and no observations; a complete observation whose trailing newline was lost is **silently dropped** by `while read`, so the one sibling that *was* degraded vanishes and the run reads “isolation held”; and a partial log — one probe subshell died before appending — reads “proven” on half the evidence. None is reachable from this file’s own writers under POSIX append semantics, so the contract is defence in depth against a future writer, a full disk or a killed subshell — never a claim that the current writers tear.

Mutation evidence (2026-08-26)

Mutation	Effect	Result
M1 — `retry_after_wellformed` accepts any non-empty value	the parse degrades to a presence check	all four carrier fixtures flip to `responsive` → **FAIL** (7 assertions at 32/32)
M2 — the decision seam reverted to the pre-fix `^2`-only rule	classifier intact but unconsulted	`decision 'all-responsive'` flips to `rc=1` → **FAIL** (4 assertions at 32/32)
M3 — drop the 429 branch entirely	every 429 is degraded	both well-formed-429 fixtures flip to `degraded` → **FAIL** (7 assertions at 32/32)
M-DATE (reviewer-authored,	the documented	pre-fix: the whole challenge exited `1` accusing the detector;

Mutation	Effect	Result
§11.4.194(6)(d)) — PATH-shim a `date` that refuses `-d`	portability branch engages	now: `NOT EXERCISED (reason=no_date_parser_on_host)` **→ honest refusal naming its cause**
M-SEAM (reviewer-authored) — delete the live call site, inline `isolation_bad=0`	live verdict bypasses the oracle	**undetected — suite fully green at every size it has been measured: 16/16 when first demonstrated, 22/22 after round-2 hardening, 32/32 as of 2026-08-26** — see “Residual risks” below

Residual risks (§11.4.6 — stated, not closed)

1. **A wedged endpoint that still answers 429-with-well-formed-`Retry-After`** classifies `responsive` and passes assertion (c). Parsing the header *narrows* this (a limiter emitting a real, positive, parseable “come back in N” is running enough code to compute and serialize one, and reset/timeout/5xx/429-with-no-header are all still caught) but does not close it. Closing it needs a semantic liveness oracle — assert the sibling still serves its own payload once its window reopens — which is outside this challenge’s bounded localhost chaos budget and is **not** claimed. Assertion (c) proves *“the sibling’s HTTP stack is alive and answering coherently under a neighbour’s load”*, never *“the sibling is fully functional”*.
2. **The live call site itself is unguarded.** Every *rule* it applies is fixture-covered, but deleting or bypassing the one `sibling_isolation_report` call — inlining a different rule, or hardcoding `isolation_bad=0` — is not detected by any test (measured: M-SEAM above left the unit test fully green at every size measured: 16/16, then 22/22, then 32/32 on 2026-08-26). Guarding it needs the real `:7185`/`:7187`/`:7189` topology under real load, which no localhost fixture can own. A grep-gate asserting the call site “looks right” is **not** the closure — that is the §11.4.201(7)(a) substring check this challenge forbids elsewhere. The honest control is the §11.4.142 review of any diff touching that line; the gap is named in-source at the call site so that review is possible.
3. **`Retry-After: 999999999`** (~31.7 years) classifies `responsive` — within the recorded operator decision, though a cap would narrow risk 1 further. Conversely a 20-digit value goes `degraded` on bash integer limits where RFC 9111 says treat overflow as max — a deviation in the conservative direction. Carried to the operator as a §11.4.66 question rather than guessed.

One detector, two consumers

`ratelimit_classify` and `ratelimit_verdict_kind` are the **single** implementation. The live run and the self-test both call them, so a mutation to the detector breaks the self-test too. A second copy inside the self-test would validate the copy, not the detector that mints live verdicts — a tautology (§11.4.249).

Mutation evidence (2026-08-21)

Each mutation was applied to the real file, run, then restored and verified byte-identical by sha256. The mutation harness itself carries a control needle: it refuses to draw any conclusion unless the file's sha256 actually changed — an earlier version of this harness silently failed to apply its mutations and reported "mutation survived" from an instrument that never fired (§11.4.201(6)).

Mutation	Effect	Result
M1 — 429 tally always returns `999`	detector always claims "engaged"	`absent` + `carrier` both flip to `engaged` → **FAIL**
M2 — 429 tally always returns `0`	detector can never see a real limiter	`enforcing` flips to `absent` → **FAIL**
M3 — probe records the response **body prefix** instead of the HTTP status code (the §11.4.201(9) field-identity regression)	a body that *mentions* `429` is read as the *status* `429`	**only** `carrier` flips (`429=12`) → **FAIL** — this is the carrier fixture earning its place

Sample-relative assertions

Assertions are made against the sample actually collected, never against the literal request count: `run_curl_status_probe` stops launching once its wall-clock budget is spent, so a hard `== 30` would turn ordinary host load into a false FAIL. The self-test runs under its own larger budget (30s, vs the 12s live-tier budget which exists to bound a genuinely-slow *real* backend) and refuses to conclude at all below `SV_MIN_SAMPLE` — a truncated sample is not evidence either way (§11.4.201(6)).

Scoping decisions

The real fanout path, `POST /api/v1/search`, is never driven by this challenge. Hammering it at any of the tested concurrency tiers would flood up to 43 **real third-party tracker sites** — risking IP bans on shared trackers, violating good-netizen practice, and doing real off-host damage that no localhost-only chaos budget can bound (§12.6/§12.11 explicitly scope host-safety to *this* host; they say nothing about the blast radius on someone else's server, and common sense fills that gap). `GET /health` stands in as "is the merge-search process up and answering" — the crash-resistance signal §11.4.85 needs — without touching the expensive orchestration path.

Followup filed: a sandboxed/mocked-tracker variant of this challenge that exercises the real search-orchestration code path (dedup, enrichment, SSE streaming) under load without ever making a real outbound tracker request — e.g. by pointing `trackers=` at a disabled/nonexistent tracker name (confirmed during authoring: `POST /api/v1/search` returns `404 Not Found` for a GET probe since it's POST-only, so this needs a real POST body to explore properly) or by standing up a local mock tracker HTTP server the merge-search's tracker registry can be pointed at for the duration of a test run.

Findings (2026-08-18, from authoring this challenge)

Authoring this scaffold immediately surfaced two real gaps — consistent with §11.4.238's mandate that automated QA be the discovery channel:

1. No rate limiting anywhere — SUPERSEDED 2026-08-26 (see "RED/GREEN polarity" above)

> **Superseded.** This finding was true when written on 2026-08-18 and is no longer. `merge_search :7187` now enforces a limiter (`x-ratelimit-limit: 120` with a `retry-after`; 33 x 429 at c=50 measured live 2026-08-26), so it PASSes assertion (b) in GREEN. `:7185` and `:7189` are still unprotected and still `SKIP` as `extension_absent`. The source-inspection record below is retained as the dated evidence it was, not as current state. Its arrival is also what surfaced BOB-163: assertion (c) read `:7187`'s own 429 as a degraded sibling.

Verified by source inspection across the whole stack — no `slowapi`/`limiter`/`throttle` import in `download-proxy/src/`, no rate-limit middleware in `qBitTorrent-go/internal/middleware/` (only `cors.go` + `logging.go` exist there) or `internal/jackettapi/` (only

`auth_middleware_test.go` + `cors_middleware_test.go` exist — no rate middleware), and no nginx/reverse-proxy service in `docker-compose.yml`.

Candidate remediation approaches (documented, not implemented — out of this task's file scope):

- A lightweight nginx-in-container reverse proxy fronting `:7185`/`:7187`/ `:7189` with `limit_req_zone` — the most portable single fix, but adds a new container to the compose stack.
- A FastAPI dependency (e.g. `slowapi`) for the merge-search service specifically, since it already sits behind uvicorn.
- A Gin rate-limit middleware for `boba-jackett` (Go), following the existing `internal/middleware/` pattern (`cors.go`, `logging.go`).
- qBittorrent's own WebUI has a documented failed-login ban list (bandwidth-shaping "Rate limiting" settings exist in the WebUI but were **not verified** during this task to cover request-rate, only bandwidth — do not assume this closes the gap without checking).

2. `boba-jackett`'s `/healthz` amplifies under a cold-start burst — FIXED + live-verified (BOB-112, task #74)

Status update (2026-08-18, task #74): the 30s TTL cache recommended in fix option 1 below was implemented as commit `91b52db` and then **live-verified with real `wrk` load-test evidence** — the followup this finding originally filed. See `docs/qa/BOB-112/summary.md` for the full before/after numbers: a §1.1 mutation reproducing this exact pre-fix code measured **97.1% client-side timeouts** (400/412 requests) at 4 threads × 100 connections × 30s against `:7189/healthz`; the real committed fix measured **0.0% timeouts** across 812,149 requests in the same window (27,049 req/s, p99 19.89ms) and collapsed upstream `Jackett.GetCatalog()` calls from one-per-request down to 2 calls in the entire 30s run. This challenge script now also ships a standing `--healthz` mode (see "Running manually" below) that re-runs a bounded version of this same check on every future invocation, so a regression that re-introduces the uncached call path trips a real `FAIL`, not just a filed finding.

The pre-fix root cause is preserved below for historical/forensic context.

Root cause: `qBitTorrent-go/internal/jackettapi/health.go:60-63` —

```
if d.Jackett != nil {
    _, err := d.Jackett.GetCatalog()
    out.JackettOk = err == nil
}
```

`HandleHealth` makes a **synchronous, uncached** call to Jackett's catalog endpoint on **every single hit** to `/healthz`. There is no cache, no timeout distinct from the Jackett client's own default, and no circuit breaker.

Observed evidence (three independent live runs, `RED_MODE=0`, 2026-08-18):

```
--- boba_jackett: Boba Jackett API :7189 ---
    reachable: HTTP 200
    tier c=10 n=50: 5xx=0 conn_fail=0  (timeout=0  other=0) 429=0
elapsed=12.40s
    tier c=25 n=50: 5xx=0 conn_fail=48 (timeout=48 other=0) 429=0
elapsed=8.50s
    tier c=50 n=50: 5xx=0 conn_fail=50 (timeout=50 other=0) 429=0
elapsed=7.17s
  [FAIL] boba_jackett: 0 5xx + 98 connection failures out of 150
requests — endpoint degraded under bounded load
```

98/150 (65%) of health-check requests timed out at the 3-second budget during this run. **Two subsequent runs against the same live process did not reproduce this** — all three tiers completed in ~2s each with zero failures once the initial burst had passed:

```
    tier c=10 n=50: 5xx=0 conn_fail=0 (timeout=0 other=0) 429=0
elapsed=2.00s
    tier c=25 n=50: 5xx=0 conn_fail=0 (timeout=0 other=0) 429=0
elapsed=1.76s
    tier c=50 n=50: 5xx=0 conn_fail=0 (timeout=0 other=0) 429=0
elapsed=2.29s
  [PASS] boba_jackett: 0/150 requests returned 5xx or failed to
connect across tiers 10 25 50
```

This is honestly reported as an intermittent, cold-state-dependent finding, not a deterministic one — per §11.4.7 (demotion requires same-conditions evidence), the recovery does NOT retract the finding; it narrows it. The trigger condition (a concurrent burst against a `/healthz` route that has not been recently exercised) is real and reproducible enough to have appeared in this task's very first live run, but this scaffold does not currently force a guaranteed-cold state (that would require restarting the `boba-jackett` container as part of a routine test run — a more invasive chaos action deliberately left out of this scaffold pending further review; see followups).

Impact if left unfixed: an attacker (or even a mis-tuned external health-monitor hitting `/healthz` too aggressively) could make the Jackett management API's own health surface appear down without ever touching Jackett itself — a genuine self-inflicted amplification vector, exactly the class of defect §11.4.85's DDoS/chaos discipline exists to catch.

Recommended fixes (documented, not implemented — `qBitTorrent-go/ internal/jackettapi/health.go` is out of this task's file scope):

1. Cache the Jackett liveness signal with a short TTL (e.g. 10-30s), refreshed by a background ticker rather than per-request.
2. Add a tight timeout + circuit breaker around the `GetCatalog` call so `/healthz` itself never blocks past ~250-500ms regardless of Jackett's real state.
3. Make `jackett_ok` best-effort / asynchronously refreshed rather than synchronously computed on the request path.

Because the running boba-jackett process fully recovered (confirmed via `podman ps` — both `jackett` and `boba-jackett` containers stayed `Up (healthy)` throughout, and a post-test manual probe returned `HTTP 200` in 0.51s), this is a **resilience/availability defect, not a crash** — the process itself never died.

How to interpret a `FAIL`

- The expected rate-limiting result as of **2026-08-26** is **`PASS` on `merge_search` `:7187`, `SKIP` on `:7185` and `:7189`** — `:7187` enforces a limiter, the other two do not, and a `SKIP` there reflects that real absence rather than a broken challenge. *(Superseded: this previously read "`SKIP` for all three endpoints … the currently-committed state", which was true only while no endpoint enforced a limiter.)* A `SKIP` on `:7187` would now be a **regression signal** — its limiter stopped firing.
- If `boba_jackett`'s crash-resistance assertion `FAIL`s occasionally, that is the tracked, real, cold-start amplification finding above — not the challenge itself being flaky. Re-run it; if it now `PASS`es, the endpoint had already been "warmed" by a prior probe (including this challenge's own earlier tiers, or another test suite/health-checker hitting it recently).
- Any `FAIL` on `merge_search` or `qbittorrent`'s crash-resistance or cross-endpoint-isolation assertions is unexpected and should be investigated per §11.4.102 (systematic debugging) before assuming it is environmental.
- `SKIP` for "endpoint unreachable" (reason `services_not_running`) means the boba stack was not running when the challenge executed — start it via `./start.sh` and re-run.

Running manually

```
# Full live run against whatever boba services are currently
        reachable at
# 127.0.0.1:7185 / :7187 / :7189.
bash challenges/scripts/ddos_resilience_challenge.sh
```

```
# RED mode — reproduce the "no rate limiting configured" defect
        state
# explicitly. As of 2026-08-26 this PASSes on :7185 and :7189 and
        FAILs on
# merge_search :7187, because RED starts FAILing for an endpoint
        the moment a
# limiter is WIRED there (the defect it reproduces is fixed) — it
        does not
# require the limiter to be stripped again.
RED_MODE=1 bash challenges/scripts/ddos_resilience_challenge.sh

# Self-validation — prove all THREE detectors distinguish their
        golden-good
# from their golden-bad fixtures: crash resistance, rate limiting
        (including
# the advertises-but-never-enforces carrier), and sibling
        responsiveness
# (including four 429s that DO carry a Retry-After whose value is
        not
# well-formed, both polarities of the isolation decision, and its
# fail-closed count contract). Throwaway local fixtures only.
bash challenges/scripts/ddos_resilience_challenge.sh --self-
        validate

# --healthz — BOB-112 regression guard (added task #74): a bounded
        2-thread
# x 20-connection x 5s wrk load (falls back to a curl-loop probe
        if wrk is
# not installed) against boba-jackett's :7189/healthz, asserting
        the
# client-observed timeout rate + throughput + server-side cache-
        refresh
# count all stay within thresholds calibrated against the real
        RED/GREEN
# evidence in docs/qa/BOB-112/summary.md. FAILs if the TTL cache
        is ever
# removed or bypassed again.
bash challenges/scripts/ddos_resilience_challenge.sh --healthz
```

Exit codes: `0` = all `PASS` (honest `SKIP`s allowed), `1` = one or more real `FAIL`, `2` = invocation error (curl itself is missing).

Followups filed

Originally documented here rather than inserted into `docs/workable_items.db` directly — this task ran in parallel with other subagents also touching shared project state, and a concurrent SQLite write from this task risked a race with theirs. Registered

into the DB (§11.4.93/§11.4.202) by the orchestrating session once the parallel batch completed — see the BOB-NNN id on each item below:

1. **Bug** (filed as **BOB-112**, **FIXED + live-verified 2026-08-18 — task #74**): `boba-jackett`'s `/healthz` synchronously called `Jackett.GetCatalog()` uncached on every hit (`qBitTorrent-go/internal/jackettapi/health.go:60-63`), causing up to 65% request timeout rates under a modest cold-start concurrent burst (measured evidence above). Fixed by commit `91b52db` (30s TTL cache); see `docs/qa/BOB-112/summary.md` for the real `wrk` before/after evidence (97.1% → 0.0% timeouts) and the standing `--healthz` regression guard this challenge now ships.
2. **Task** (filed as **BOB-111**): configure real rate limiting for boba's three public HTTP endpoints (`:7185`, `:7187`, `:7189`) — none exists today. Candidate approaches documented above.
3. **Task** (NOT filed as a separate BOB-NNN this round — remains an open scoping note here, distinct from the orchestrating session's filed batch): extend `ddos_resilience_challenge.sh` (or add a sibling challenge) to exercise the real `POST /api/v1/search` fanout path via a sandboxed/mocked-tracker setup, without ever touching real third-party tracker sites.
4. **Task** (filed as **BOB-114**, **CLOSED 2026-08-21**): add a golden-bad synthetic fixture to `--self-validate` that validates the rate-limiting detector itself. Delivered with three fixtures (enforcing / absent / advertises-but-never-enforces carrier), the full polarity truth table, a shared single-implementation detector, and automatic execution on every invocation — see "Self-validation" above for the RED and the M1/M2/M3 mutation evidence.
5. **Task (test-type matrix, see `docs/testing/test_type_matrix.md`)** (filed as **BOB-109** scaling / **BOB-110** UX): scaling-class and UX-class test coverage are both fully absent from boba's mandated test-type matrix — filed as separate followups there.
6. **Task** (filed as **BOB-113**, **CLOSED 2026-08-18 — task #75/#74**): `wrk` was not installed on the development host (only `ab` was) — `scripts/install-dev-tools.sh` (commit `c7dfdde`) added a dev-tooling installer; it was run live during task #74 (`wrk 4.2.0 [epoll]` and `hey` confirmed `ALREADY-PRESENT`/installed, `ab`/`curl-loader` detected, `siege` an honest documented `SKIP`).